

Building a Python Interpreter

1. Project Overview

The purpose of this project was to build a simple Python interpreter that supports variable assignments, arithmetic operations, print statements, multiple statements, whitespace, and semicolons.

2. Structure of the Interpreter

The interpreter is structured as a pipeline made up of five main files: **lexer.py**, **ast_nodes.py**, **parser.py**, **interpreter.py**, and **main.py**. Together, these files move the program from source code to tokens, then to an Abstract Syntax Tree (AST), and finally to executed output. The following subsections summarize the key design decisions behind these components and explain how they work together.

2.1 Key Design Decisions

The overall design of the **lexer** was intentionally kept simple so that it would be easier to implement and make compatible with other parts of the project. One of the most important decisions in the lexer is that we made sure to have a list of keywords so that normal identifiers and variables can be separated from keywords defined by the programming language. As for the main loop of the tokenize function, we chose to use a while function; this ensures that any length of code can be inputted, which makes our project far more flexible. Additionally, within this loop we made it so that if an unexpected character is inputted, then the lexer will raise an error and show exactly which character caused the error.

The **AST nodes** were kept simple and focused on the assignment requirements. Statement nodes and expression nodes were separated to make the structure easier to understand. Python dataclasses were also used to reduce repetitive constructor code and make the AST easier to print and inspect during testing.

The **parser** was organized into small methods that match the grammar of the language, such as *parse_assignment()*, *parse_print()*, *parse_expression()*, *parse_term()*, and *parse_factor()*. This made the code easier to read, test, and debug. The *consume()* method checks that tokens appear in the expected order, which helps catch syntax errors and ensures that statements follow the required structure, including semicolons.

For the **interpreter**, the visitor pattern is used here to prevent having one long conditional block since each node type requires its own method. Later, new node types can be easily added without having to modify the existing code. The dispatcher automatically gets the correct method by using the class name of the node in the *getattr()* function of Python. When evaluating binary operations, expressions can be nested within each other and thus the recursion applies. We also included code to handle division by zero or undefined variables to prevent the program from crashing without warning or displaying an error message.

Finally, we made **main.py** small on purpose—it only invokes each part in succession; it doesn't have any logic in itself. Each component had been built and tested individually, so the only work that needs to be done in *main.py* is the linking of the components; this helped to isolate any problems in the integration phase when testing the *main.py* was being conducted during Milestone 3.

2.2 Summary of How It Works

The lexer file, **lexer.py**, contains two classes: the “token” class and the “Lexer” class. The token class defines the tokens that the lexer returns, each token containing the type and value of said token, and the lexer class contains a function that returns all the tokens contained within the inputted code. The function in question, “tokenize,” runs a loop that makes a token for each separate lexeme in the code. For example, if the number 5 is detected, the code will create a token of the “token” class that has the type of “NUMBER” and the value of 5.

The AST nodes are defined in **ast_nodes.py** and represent the program’s structure after parsing. *ProgramNode* stores all statements in the order they should be executed. *AssignNode* represents variable assignments, and *PrintNode* represents print statements. Expression nodes include *BinOpNode* for arithmetic operations, *NumberNode* for numeric values, *VarNode* for variable names, and *UnaryOpNode* for unary operations such as negative numbers. Together, these node classes provide a structured form of the source program that the interpreter can evaluate after the parser finishes building the AST.

The parser is implemented in **parser.py**. It takes tokens from the lexer and builds the AST using the classes from **ast_nodes.py**. The parser identifies assignment and print statements, creates the correct AST nodes, and parses arithmetic expressions using separate methods for expressions, terms, and factors so multiplication and division are handled before addition and subtraction.

The interpreter, in **interpreter.py**, walks through the AST built by the parser and executes it using the visitor pattern. A visit() method checks each node's type and calls the matching function, like visit_AssignNode or visit_BinOpNode. Variable values are stored in a dictionary as assignments run. In arithmetic, the interpreter first processes the left and right sides of an expression and then applies the operator so that it can correctly process expressions involving nested operators.

Finally, **main.py** starts the whole pipeline, it sends the full source code to the lexer to get the tokens, the tokens to the parser to create the AST, and the AST to the interpreter to run it and get output.

3. Example Programs and Outputs

The following table summarizes the example programs and outputs used to test the interpreter. These examples align with the project milestones for lexical analysis, parsing, and interpretation, as well as the required language features.

Test	Purpose / Description	Example Program	Output / Result
Milestone 1: Lexer	Confirms that the lexer converts source code into tokens.	let x = 10 + 5; let y = x * 2; print(x); print(y);	Token sequence includes LET, ID, ASSIGN, NUMBER, PLUS, SEMI, and PRINT. See Figure 1.
Milestone 2: Parser / AST	Confirms that the parser builds the expected AST from tokenized input.	let x = 10 + 5; print(x);	Output shows a <i>ProgramNode</i> with an <i>AssignNode</i> and <i>PrintNode</i> . See Figure 2.
Milestone 3: Interpretation	Confirms that the interpreter executes the parsed AST.	let x = 10 + 5; let y = x * 2; print(x); print(y);	Expected output: 15 30 See Figure 3.
Required-Features Test	Confirms subtraction, division, multiple sequential statements, whitespace, and semicolons.	let a = 21 ; let b = 4; let c = a - b ;	Expected output: 17

		<pre>let d = a / b ; print(c); print(d) ;</pre>	5.25 See Figure 4.
--	--	---	-----------------------

```
Token types:
['LET', 'ID', 'ASSIGN', 'NUMBER', 'PLUS', 'NUMBER', 'SEMI', 'LET', 'ID', 'ASSIGN', 'ID', 'MUL', 'NUMBER', 'SEMI', 'PRINT', 'LEFTPAREN', 'ID', 'RIGHTPAREN', 'SEMI', 'PRINT', 'LEFTPAREN', 'ID', 'RIGHTPAREN', 'SEMI']
Milestone 1 lexer test passed.
```

Figure 1. Milestone 1 lexer test output showing that the sample program was converted into the expected token sequence.

```
AST:
ProgramNode(statements=[AssignNode(var_name='x', expr=BinOpNode(left=NumberNode(value=10), op='PLUS', right=NumberNode(value=5))), PrintNode(expr=VarNode(name='x'))])

AST checks:
Milestone 2 parser/AST test passed.
```

Figure 2. Milestone 2 parser/AST test output showing that the parser created the expected ProgramNode, AssignNode, BinOpNode, and PrintNode structure.

```
Expected output:
15
30

Actual output:
15
30

Milestone 3 interpreter test passed.
```

Figure 3. Milestone 3 interpreter test output showing that the sample program executed correctly.

```
Expected output:
17
5.25

Actual output:
17
5.25

Extra required features test passed.
```

Figure 4. Required-features test output showing that subtraction, division, whitespace, semicolons, and multiple statements worked correctly.

4. Group Contributions

Team Member	Primary Contribution
LeBrea Franklin	Lexer
Aven Pacha Pinto	AST Nodes and Parser
Veera Venkata Naga Surya Jaya Sa Padala	Interpreter and main.py integration
All group members	Testing, report, and demo